

Segment wise Theory+ Previous Theory Solve

Segment 04:

1. Define encryption and decryption.

Encryption. is a security technique used to protect sensitive data stored in database It involves converting data into a secure, unreadable format, which can only be deciphered with the appropriate decryption key. Here are some key point about encryption in DBMS.

Decryption is the process of converting encrypted data back into its original, human-readable form. This is typically done to access and use sensitive information stored in a database while ensuring data security.

2. Determine why roles are needed in case of privileges ?Explain.

Privilege refers to the rights or permission grants to users or rules to perform various kind of action in Database.Roles are needed to simplify the management of **privileges**. Instead of assigning permissions one by one to every single user, you can group a set of related privileges into a **role** and then assign that role to users.There are some common type in privilege.

Select → Allows users to read data from specific table.

Insert → Permits users to add new record to a table in database.

Delete → permits to delete data from a table.

Update→ Grant the ability to modify in existing data from database.

Drop→ Allows user to delete database obj like table etc.

Grant→ Allows users to assign pervilegs. Revoke→ Enable removal of previous privileg. Execute, Index halka bekka krbo.

3. How the trigger does ensure integrity on the database ?Explain with example.

A database **trigger** ensures integrity by automatically executing a set of actions whenever a specific event, like an INSERT, UPDATE, or DELETE, occurs on a table. This allows you to enforce complex **business rules** and maintain data consistency that standard constraints can't handle. Triggers act like automated watchdogs for data. While constraints like **PRIMARY KEY** or **FOREIGN KEY** enforce structural integrity, triggers enforce *procedural* integrity. They are essential when the integrity rule involves more than one table or requires complex logic.

- **Enforcing Complex Business Rules:** A trigger can check conditions before allowing a data modification. For example, it can prevent an order from being placed if the customer's credit limit would be exceeded.
- **Maintaining Data Consistency:** They can automatically update related data in other tables.
- **Creating Audit Trails:** A trigger can automatically log changes to a separate audit table whenever sensitive data is modified.

Example:

Let's say we have two tables: **products** and **sales**.

products table: has columns **product_id** and **quantity_in_stock**.

Sales table: has columns **sale_id**, **product_id**, and **quantity_sold**.

The Integrity Rule: The **quantity_in_stock** in the products table must always *reflect the correct* inventory after a sale is made.

4. What are the different types of referential integrity constrain, and how can they be used to ensure the integrity of data in a relational database ?

Referential integrity constraints are rules that protect the links between related data in a database. They ensure that you don't end up with "orphan" records, like an order that belongs to a non-existent customer.

1. RESTRICT

This is the default and most common rule. The database stops you from deleting a parent record if it has any linked child records.

- **How it works:** If you try to delete a customer who has existing orders, the database will show an error and say "No."

2. CASCADE

This rule links the parent and child records together completely. What happens to the parent also happens to the child.

- **How it works:** If you delete a customer, all of their orders are automatically deleted too.

3. SET NULL

This rule breaks the link by setting the child's linking field to `NULL` (empty). This is only possible if the child's linking column is allowed to be empty.

- **How it works:** If you delete a customer, the `customer_id` field in their `Orders` is set to `NULL`. The order records still exist, but they are no longer tied to that specific customer.

4. SET DEFAULT (The "Re-assign" Rule)

This rule reassigns the child record to a default parent if the original parent is deleted. This requires the linking column to have a default value.

- **How it works:** If you delete a customer, their orders could be automatically reassigned to a default "Guest" or "Archived" customer account.

5. Define Referential integrity constrain with example.

It is a database concept that ensures the consistency and accuracy of data by enforcing relationships between tables. It means that relationships between tables are maintained and that foreign key values in one table must match the primary key values in another table.

Examples:

Suppose I have two tables in a relational database "Orders" and "Customers". The "Orders" table has a foreign key column called "CustomerID" that references the primary key column "customer ID" in the "Customers" table. Referential integrity in this context means that every "CustomerID" value in the "Orders" table must correspond to a valid "CustomerID" in the "Customers" table.

6 – 04 types of integrity with example.

Segment 05:

6. Explain the difference between dependency preservation and lossless decomposition with examples.

Property	Dependency Preservation	Lossless Decomposition
Meaning	After decomposition, all functional dependencies (FDs) from original relation can still be enforced without recombining the tables.	After decomposition, when we join the decomposed tables , we get back exactly the original relation (no spurious tuples).
Focus	Maintaining constraints (FDs) easily.	Maintaining data correctness after join.
Why Important?	To avoid recomputing FDs with joins when checking constraints.	To ensure no extra (spurious) tuples are generated.
Achieved By	Choosing decomposition where every FD is preserved in at least one relation.	Using a decomposition that satisfies the lossless join condition .
Trade-off	Sometimes you can't have both together (lossless + dependency preservation) for all cases.	Lossless is mandatory , dependency preservation is desirable .

7. Explain lossless decomposition and lossy decomposition with an example.

Feature	Lossless Decomposition	Lossy Decomposition
Definition	Decomposition where no data is lost	Decomposition where some data may be lost or incorrect
Join Result	Join gives back the original relation exactly	Join gives extra or wrong data
Condition	Common attribute must be a superkey in at least one table	Common attribute is not a superkey

Feature	Lossless Decomposition	Lossy Decomposition
	Used in normalization to avoid redundancy safely	Leads to data inconsistency or error Employee(EmpID, Name, Dept)
Goal	Employee(EmpID, Name, Dept) ☐ R1(EmpID, Name) ☐ R2(EmpID, Dept) EmpID is the common attribute EmpID is a key in the original table So, this is a lossless decomposition.	R1(Name, Dept) R2(EmpID, Dept) Here: Common attribute is Dept Dept is not a key This is a lossy decomposition.

8. What do you understand by FD ? Write down and explain the types of FD...

A Functional Dependency (FD) is a relationship between two attributes in a relational database. It describes how one attribute uniquely determines another.

Types of Functional Dependency .

1.Trivial Functional Dependency : A dependency $X \rightarrow Y$ is trivial if **Y is a subset of X**.

Example:

`{StudentID, Name} → Name`

2. Non-Trivial Functional Dependency : A dependency $X \rightarrow Y$ is non-trivial if **Y is not a subset of X**.

Example:

`StudentID → Name` Name is not part of StudentID. This is **non-trivial** and useful in design.

3. Partial Functional Dependency: Y is **partially dependent** on X if Y depends on **part of a composite primary key**.

Example:

If: Roll, Subject → Marks

And also: Roll → StudentName

Then StudentName is **partially dependent** on the primary key {Roll, Subject}.

4. Transitive Dependency: A dependency $X \rightarrow Z$ exists indirectly via $X \rightarrow Y$ and $Y \rightarrow Z$.

Example:

StudentID $\rightarrow$ Department
Department $\rightarrow$ HOD Then: StudentID $\rightarrow$ HOD (Transitive)

5. Multivalued Dependency: When one attribute in a table determines **multiple independent values** of another attribute.

Example:

StudentID $\twoheadrightarrow$ PhoneNumber
StudentID $\twoheadrightarrow$ Email

9. What do you mean by normalization ? why this is important ?

Normalization is the process of organizing data in a relational database to **minimize redundancy** and **ensure data integrity**. **Normalization** helps in creating a clean, logical, and efficient database structure.

Avoid Redundancy	Prevents storing the same data repeatedly (e.g., department name per student)
Prevent Anomalies	Solves update, insert, and delete anomalies
Ensure Consistency	Ensures related data stays synchronized across tables
Better Organization	Makes the database easier to manage, understand, and expand
Storage Optimization	Saves space by eliminating duplicate values
Improved Query Performance	Smaller, focused tables allow for faster queries and indexing

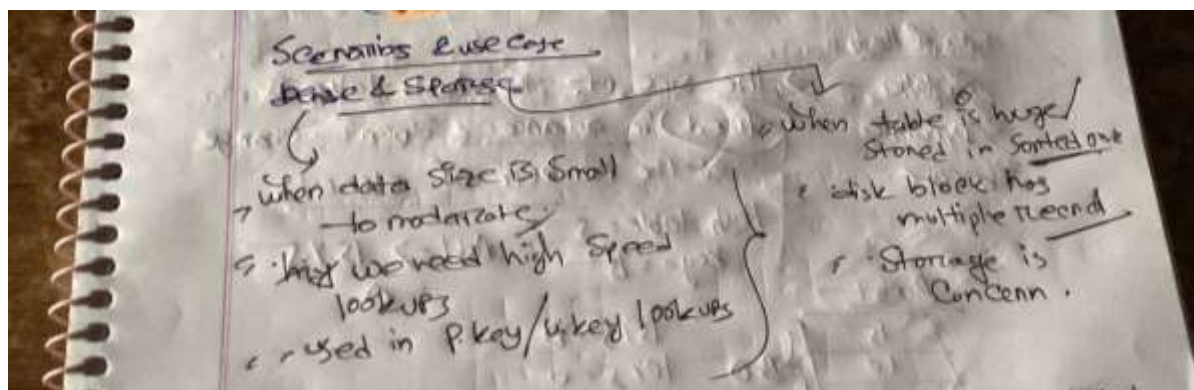
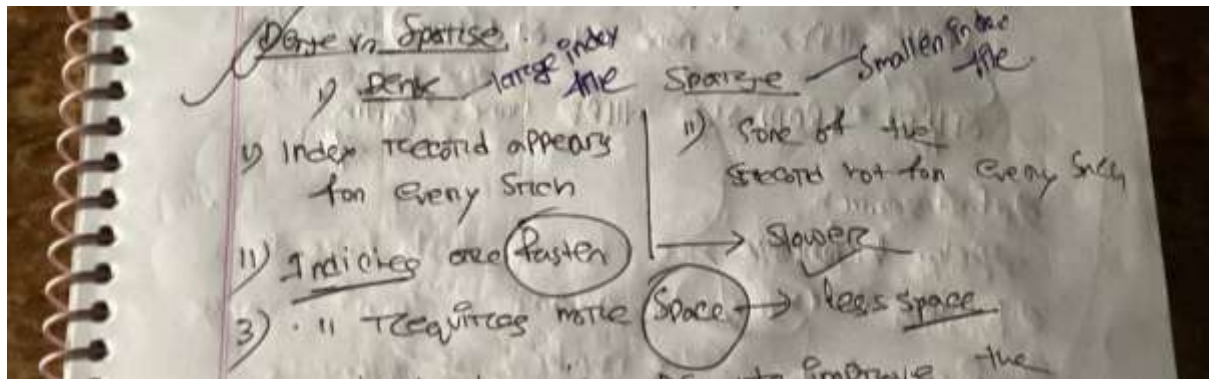
use to reduce data redundancy. The first stage of decomposition to do the normalization is called Normal form (NF). Normalization is an essential part of product information management preventing data from being replicated in two tables at the same time or unrelated product data being gathered together in the same table. In addition, normalization helps to streamline your data, simplifying your database and making it more concise.

-> Every Types of Normalization – 1NF , 2NF, 3NF,BCNF Deffination.

Segment 06 :

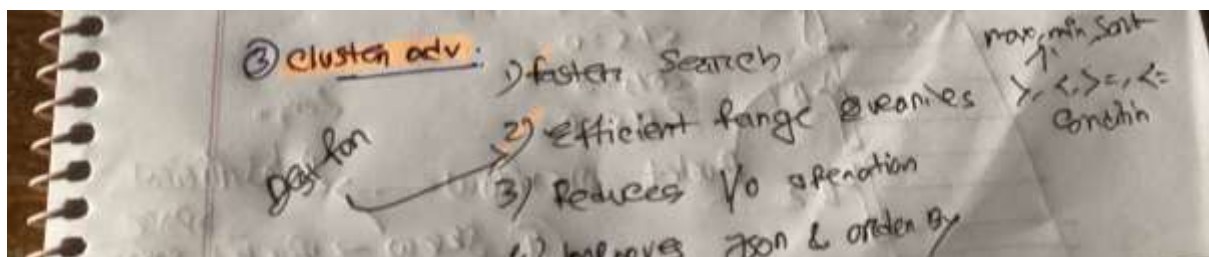
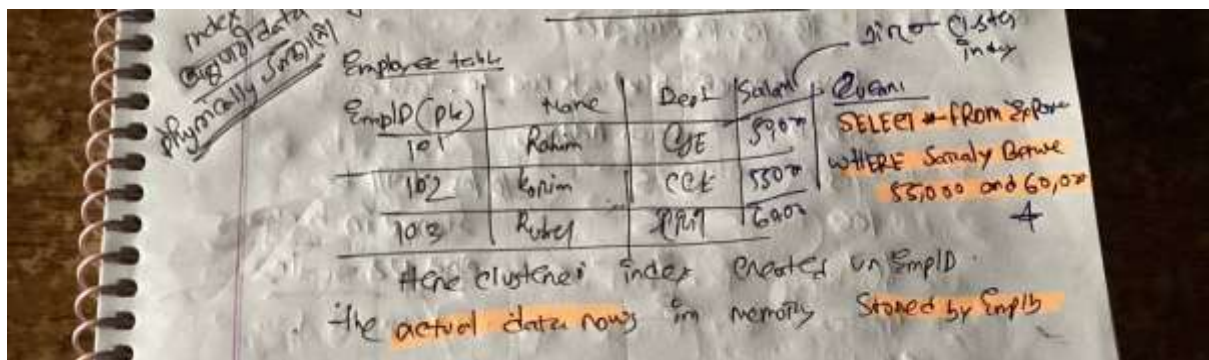
1. Compare and contrast dense and sparse indexing techniques. Provide sceanarios where each would be more effective.

Write diff between dense and sparse then compare scenarios and use case of both for more effective.



2. Discuss the role of clustered indexes in improving query performance with example.

Define Clustered Index then give an example with query, then give advantage of this index.



3. How to choose indexing or hashing technique ? justify.

Feature	Indexing	Hashing
Purpose	Efficient range and sorted access	Efficient exact match lookup
Data Organization	Keeps data in sorted or semi-sorted order	Uses hash function to compute address
Ideal for	Range queries (e.g., >, <, BETWEEN)	Exact match queries (e.g., = only)
Structure	Tree (e.g., B+ Tree) or Dense/Sparse index	Hash table with buckets and chaining
Performance	Slower for equality, better for range	Faster for equality, bad for range

Use Hashing for fast, exact key-based searches.

Use Indexing for range queries, sorted data access, and complex conditions.

Justify: B+ Tree indexing allows you to traverse keys in sorted order efficiently.

Hashing provides **constant time** complexity $O(1)$ for exact match.

4. How can indexes help performance ?

- Faster Data Retrieval (Search)
- Improves Performance of WHERE Clauses `SELECT * FROM orders WHERE order_date = '2025-07-19';`
- Speeds Up JOIN Operations
- Faster Sorting (ORDER BY)
- Speeds Up GROUP BY and Aggregate Functions

5. what is the difference between closed hashing and open hashing in DBMS and which one is generally more efficient.

Criteria	Closed Hashing (Open Addressing)	Open Hashing (Separate Chaining)
Memory usage	More compact, uses only one array	Needs extra memory for linked lists
Deletion complexity	Higher (due to tombstones)	Lower (simple list deletion)
Performance at high load	Poor	Better
Load factor	Must be < 1	Can be > 1
Cache performance	Better (all data in one array)	Poorer (linked structure, pointer chasing)
Implementation	Simpler for small datasets	More flexible for dynamic datasets

❑ **Closed Hashing (Open Addressing)** is **more efficient** when:

- Memory is tight
- Dataset is small or moderately sized

Criteria	Closed Hashing (Open Addressing)	Open Hashing (Separate Chaining)
Cache performance is important (e.g., in-memory lookups)		
☐ Open Hashing (Separate Chaining) more efficient when: • There are frequent insertions/deletions • The data grows dynamically • You want to avoid performance degradation at high load factors		

6. what are the advantage and disadvantages of using hash structures?

Advantages of Hash Structures

1. **Very Fast Search**
 - You can find data quickly, usually in 1 step.
2. **Great for Exact Match**
 - Perfect when you know exactly what you're looking for (like a student ID or username).
3. **Easy to Use**
 - The method is simple and easy to work with.
4. **Saves Time**
 - Less time is needed to search compared to other methods like trees or lists.

Disadvantages of Hash Structures (Easy Way)

1. **No Range Search**
 - You can't find data between two values (like 100 to 200) easily.
2. **Collision Problem**
 - Sometimes two values go to the same place – this needs extra work to fix.
3. **Bad for Changing Data**
 - If data grows or shrinks a lot, performance may get worse.
4. **Wastes Space**
 - Sometimes memory is not used properly if the table is too big.
5. **Needs Good Hash Function**
 - If the hash function is bad, it becomes slow.

- 2) What are the causes of bucket overflow in a hash file organization? What can be done to reduce the occurrence of bucket overflows?
- 3) Why is a hash structure not the best choice for a search key on which range queries are likely?

7.

See previous Solve Sp 23.

8. Open Hash, Close Hash, Bucket Overflow Solution, WHY Bucket Overflow.

Segment 07:

Do you think conflict serializability and view serializability will be enough to make schedule recoverable and cascade less? justify with example.

No, conflict serializability and view serializability alone are not enough to guarantee that a schedule is recoverable or cascadeless. These serializability concepts **ensure correctness** in terms of **consistency**, but not in terms of **recoverability or cascading rollbacks**.

- **Conflict/View Serializability** ensures that the **final outcome** of a schedule is **equivalent to some serial schedule** (i.e., consistent).
- But **recoverability** and **cascadelessness** concern **when a transaction can commit** and **what data a transaction reads**:
 - **Recoverable Schedule**: A transaction can only commit **after** the transactions it depends on have committed.
 - **Cascadeless Schedule**: A transaction only reads values **written by committed** transactions.

Neither **conflict** nor **view serializability** prevents a schedule from reading **uncommitted (dirty) data**, so **cascading rollbacks** or **unrecoverable situations** are still possible.

Let's consider this **conflict-serializable** schedule that is **not recoverable**: For schedule S

T1: write(A)

T2: read(A)

T2: commit

T1: abort

☐ **Conflict Serializability?**

- There's only one conflict: $T1 \rightarrow T2$ (write-read conflict).
- This graph has no cycle $\rightarrow$ conflict serializable.

☐ **Recoverable?**

- **T2 reads uncommitted data** from T1 and **commits before T1**.
- If T1 aborts $\rightarrow$ T2 has used invalid data $\rightarrow$ schedule is **not recoverable**.

□ Cascadeless? ✕

- T2 reads value from T1 before T1 commits → **not cascadeless**.

Explain how a transaction will be accomplished through different states. Also illustrate which properties are mandatory in terms of DB transactions.

Explain the distinction between the terms serial schedule vs serializable schedule. ***

- **Serial Schedule:**
Transactions are executed one after another, without interleaving. E.g., T1 completes entirely before T2 starts.
- **Serializable Schedule:**
Transactions may interleave, but the outcome is **equivalent to some serial execution**. It ensures **consistency** like a serial schedule but improves **performance**.

Example:

- Serial: T1 → T2
- Serializable: T1: read(A), T2: read(B), T1: write(A), T2: write(B)
(if there are no conflicts, this may be serializable)

what is cascadeless schedule ? why cascadelessness of schedules desirable?

- **Cascadeless Schedule:**
A schedule where a transaction **only reads data written by committed transactions**, i.e., **no dirty reads**.
- **Why desirable?**
 - Prevents **cascading rollbacks**.
 - Ensures **isolation and recovery** is easier.
- **Example of a cascading schedule:**

```
pgsql
CopyEdit
T1: write(A)
T2: read(A)
T1 fails → T2 must roll back too ⇒ cascading rollback
```

- **When are non-cascadeless schedules allowed?**
 - In **high-performance systems**, for **higher concurrency**, sometimes non-cascadeless schedules are tolerated, provided proper recovery mechanisms (like logging, checkpoints) exist.

Dirty read, blind write. Dirty Read:

Definition: A **Dirty Read** occurs when a transaction reads **uncommitted data** written by another transaction. If the writer transaction **rolls back**, the reader has already used **invalid (dirty) data**. It can lead to **inconsistent or incorrect results**, because the read was based on changes that never officially "happened".

Example:

Let's say:

- **T1:** Updates balance of account A
- **T2:** Reads balance of account A (before T1 commits)

```
T1: write(A = A - 500) → A = 1000 → 500
T2: read(A)           → reads 500 (dirty)
T1: abort             → A reverts to 1000
```

Now T2 has used the value 500, which was never actually committed. This is a **dirty read**

Blind Write:

Definition: A **Blind Write** occurs when a transaction **writes a value to a data item without reading it first.**

It can overwrite useful or valid data

Often seen in write-only transactions like log inserts or mass updates.

Can cause **lost update problems** in concurrency control.

Example:

```
T1: write(A = 200) → (does not read the old value of A)
```

If A was previously 1000 or 500, **T1 doesn't care**, it just sets it to 200 blindly.

Segment 08:

1. Explain the trade offs between replication and fragmentation.

Aspect	Replication	Fragmentation
Availability	High (due to multiple copies)	Lower (depends on fragment location)
Consistency	Harder to maintain	Easier (less duplication)
Read Performance	Fast (data closer to users)	Can be fast (if data is well partitioned)
Write Performance	Slower (writes must sync across nodes)	Faster (writes only to relevant fragment)
Storage Efficiency	Lower (data is duplicated)	Higher (data is not repeated)
Complexity	High (due to synchronization)	High (due to query reconstruction)

Many distributed systems use **a hybrid of replication and fragmentation**, such as:

- **Fragment-first, replicate later:** Partition data into fragments, then replicate the fragments as needed.
- **Geo-partitioning:** Fragment based on geography, then replicate locally within each region.

2. What is the role of logging in DB Recovery?

3. Discuss two techniques for resolving System failures in database system. 07

When a system failure occurs, data in volatile memory is lost. But we can recover the db to a consistent state using 3 techniques:

1. Check pointing
2. Log based
3. Shadow paging

* Exam - Q. Explain any two of the above. 2/4

1. Check pointing → Checkpointing is a technique where the system periodically saves the current states of the db and the active transactions to the disk. In the event of a failure, recovery starts from the most recent checkpoint rather than from the beginning, which significantly reduces recovery time. Together, this method ensure that the system can recover from crashes efficiently and maintain data integrity.

3. Shadow-paging → Here, a copy (or shadow) of the db page is maintained. Instead of modifying the original data directly, changes are made to the shadow page. Once the transaction is successfully completed, the page map is updated to point to the new pages. If a failure occurs, the system can fall back to the original pages, ensuring data remains consistent without the need for undo or redo operations.

4. Importance of Distributed Databases. Write the application.

Distributed databases are collections of data that are stored across multiple physical locations—either on different computers within the same network or across various geographical regions. These databases are managed by a central database management system (DBMS) that ensures data consistency, availability, and reliability across all nodes.

1. Improved Reliability and Availability:
 - If one node fails, others can continue functioning. This fault tolerance ensures high availability.
 - Data replication across nodes ensures that there is no single point of failure.
2. Scalability:
 - You can easily add more servers or nodes to accommodate growing data and user demands.
 - Supports both vertical and horizontal scaling.
3. Faster Data Access:
 - Data can be stored closer to where it is most frequently used (data localization), reducing latency.
 - Users from different regions experience quicker access times.
4. Load Balancing:
 - Workload is distributed among different servers or nodes to avoid overloading any single system.
 - This improves overall system performance.
5. Cost Efficiency:
 - Can utilize commodity hardware in distributed systems instead of investing in a large central server.
 - Cloud-based distributed databases offer flexible pricing models.
6. Support for Big Data and Global Applications:
 - Essential for applications requiring real-time processing of large volumes of data.

- Enables multinational companies to store and access data locally while maintaining a unified view.

Application Example: Global E-Commerce Platform

Scenario:

An international e-commerce company (like Amazon) wants to manage customer data, product inventories, and order information across multiple countries.

How a Distributed Database Helps:

1. Customer Experience:
 - User data is stored on servers closer to the customer's location, enabling faster page loads and personalized content.
2. Inventory Management:
 - Real-time inventory levels are synced across regional warehouses. If one location runs out, another nearby can fulfill the order.
3. Transaction Processing:
 - Orders and payments are processed regionally for speed and compliance, but a central system reconciles and tracks global operations.
4. Fault Tolerance:
 - If a data center in Europe goes offline, traffic is rerouted to another region, ensuring continued service.
5. Data Consistency:
 - Even though the data is distributed, the database system uses protocols like eventual consistency or strong consistency (depending on the need) to ensure accuracy across the system.

Technologies Used:

- Databases: Google Cloud Spanner, Amazon DynamoDB, Apache Cassandra, MongoDB Atlas.
- Orchestration: Kubernetes for managing containerized services.
- Monitoring Tools: Prometheus, Grafana, ELK Stack.

5. Explain the purpose of checkpoint mechanism. How often should checkpoints be performed given an example.

A **checkpoint** is like a "save point" in a database.

Just like when you're playing a video game and you hit a save point — if your system crashes, you can **start from the last save**, not from the beginning.

In databases, a checkpoint means saving:

- All recent changes in memory (RAM)
 - Log data
 - Transaction info
- ...to the disk, so the database can **recover faster** after a crash.

Main Purpose of Checkpoints:

1. **Faster Recovery:**
 - After a crash, the system doesn't have to go through all old logs.

- It starts from the **last checkpoint** instead, saving time.
- 2. **Reduce Log Size:**
 - Logs grow over time; checkpoints allow older log entries to be safely deleted.
- 3. **Data Consistency:**
 - Ensures that all data written before the checkpoint is safely stored on disk.
- 4. **Less Work for Recovery Manager:**
 - Easier to identify which transactions to redo or undo.

How Often Should Checkpoints Be Done?

It depends on system usage, but here's a **simple rule**:

The more transactions you process, the more frequent your checkpoints should be.

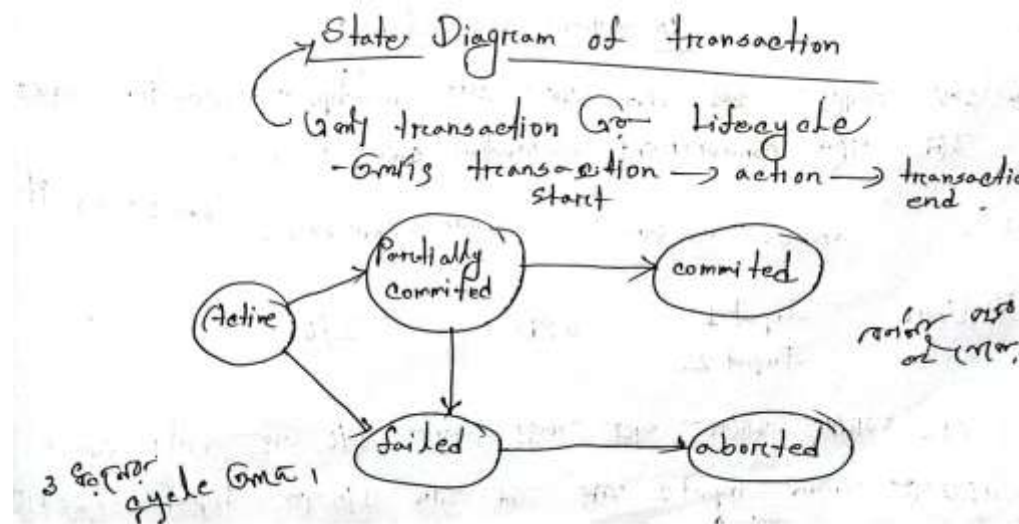
Example:

- If your system handles **thousands of transactions per minute**, do a checkpoint **every few minutes** (e.g., every 5 minutes).
- If your system is low-volume, every **15–30 minutes** may be enough.

Purpose

Save current state for fast recovery

6. Draw the state diagram of a transaction and discuss its state.



Active: The initial state, the transaction stays in this state while it is executing.

Partially committed: After the final statement has been executed.

CS CamScanner

Failed: After the discovery that normal execution can no longer proceed.

Aborted: After the transaction has been rolled back and the database has been restored to its state prior to the start of the transaction.

Committed: After successful completion.

7. What is it useful to have replication r fragmentation of data? Explain.

Replication means keeping **copies of the same data** on **multiple servers**. Fragmentation means **splitting the data into parts** and storing different parts on different servers.

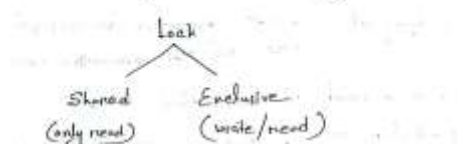
Feature	Replication	Fragmentation
What it does	Makes copies of data	Splits data into pieces
Useful for	Reliability, speed, backup	Performance, storage, localization
Real-life use	Video streaming, backups	Distributed storage, search engines

8. Describe how a typical lock manager is implement. Why must lock and unlock be atomic operations.

- Construct the timestamps assigned to restarted transactions when timestamps are use for deadlock prevention versus when timestamps are used for concurrency control.**
- When compatibility matrix use in database what type of problem will created and Explain how we resolve this problem . ******

A **compatibility matrix** (also called a **lock compatibility matrix**) is used in **concurrency control** to determine whether **two database operations (transactions)** can **work on the same**

In two modes, a data item may be locked.



	S	X
S	True	False
X	False	False

S → Shared
X → Exclusive

What Problems Can Be Created?

1. Deadlock

- Two transactions wait forever for each other to release locks.
- Example:
 - Transaction A has a lock on Row 1 and wants Row 2
 - Transaction B has a lock on Row 2 and wants Row 1
 → Both are stuck.

2. Blocking or Waiting

If one transaction locks data with **Exclusive Lock (X)**, other transactions must **wait**, causing delays.

3. Reduced Performance

- Too many locks or waiting leads to **slow performance**, especially with many users.

How to Resolve These Problems

1. Deadlock Detection and Resolution

- DBMS checks for deadlocks and cancels (rolls back) one of the stuck transactions.
- Example: In SQL Server, deadlock victim is chosen and the other continues.

2. Lock Timeout

- If a transaction waits too long, it's **automatically cancelled** to avoid long delays.

3. Using Lock Granularity Wisely

- Use **row-level locks** instead of table-level locks to reduce blocking.

4. Lock Compatibility Control

- Use the matrix to allow only **safe combinations** of locks (like multiple shared reads but no read + write at the same time).
- **5. Optimistic Concurrency Control** -- Instead of locking right away, transactions proceed and only check for conflicts at the **end** (used when conflicts are rare).